

COM3021 – Data Science at Scale

Distributed Architectures - Replication

DR HUGO BARBOSA



Agenda

- Past

How can we design efficient data storage solutions?

- This week

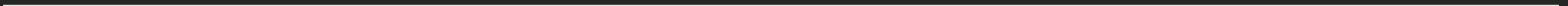
How can we deploy such storage systems?

Introduction

- Up to now we abstracted the details of where things were being stored/computed.
 - Now we will focus on aspects of **distributed** data solutions (i.e., when multiple machines and/or storage devices are involved)
 - Why?
 - What if your load increases beyond what a single machine can handle?
 - What if the machine goes down?
-

Introduction

- Distributed systems can range from two machines connected to the same PSU to global Content Delivery Networks (CDNs)



Netflix CDN (estimated)

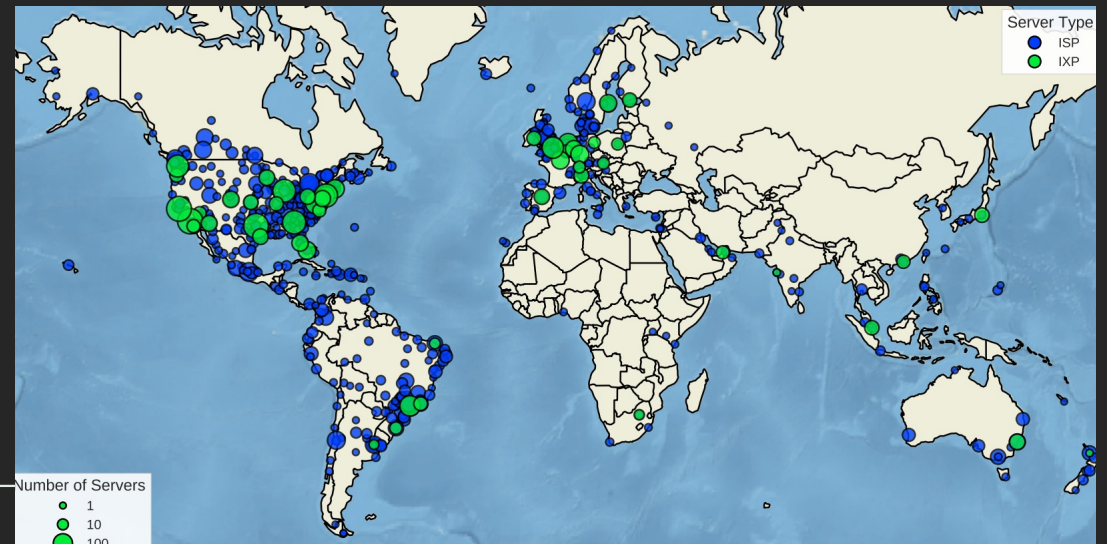
2016

- 4,669 servers
- 243 locations



2018

- 8,492 servers
- 578 locations



Scaling

- Vertical scaling (i.e., scaling up or shared-memory architectures)

The costs do not scale linearly

Limited fault tolerance

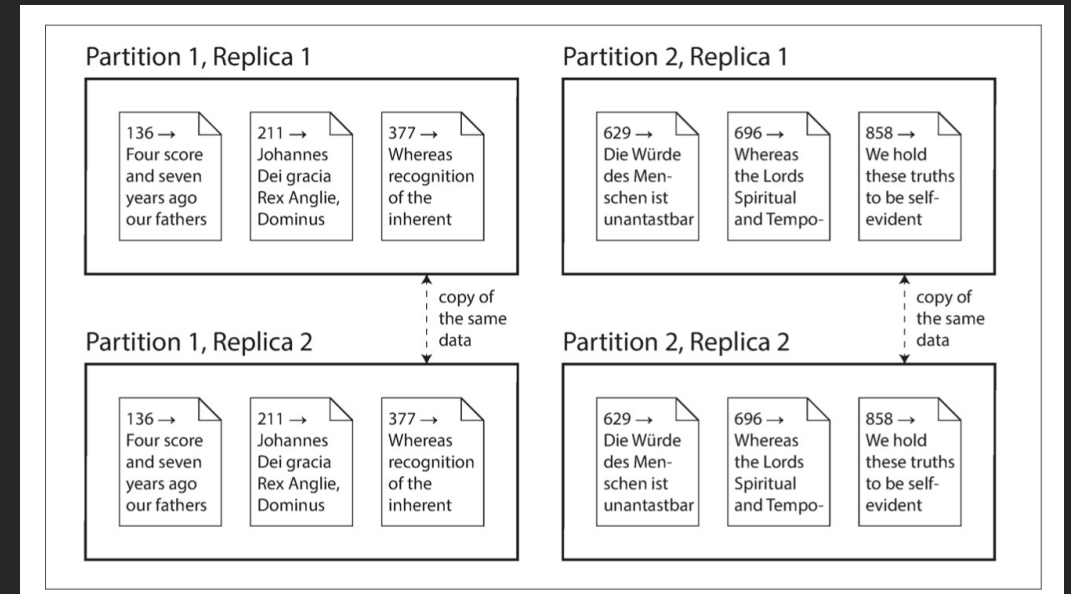
- Horizontal scaling (i.e., scaling out or shared-nothing architectures)

Costs can scale better

Better fault tolerance

Distributing data across multiple nodes

- Replication
Keeping a copy of the data in each node
- Partitioning
Different nodes will store subsets of the data called partitions



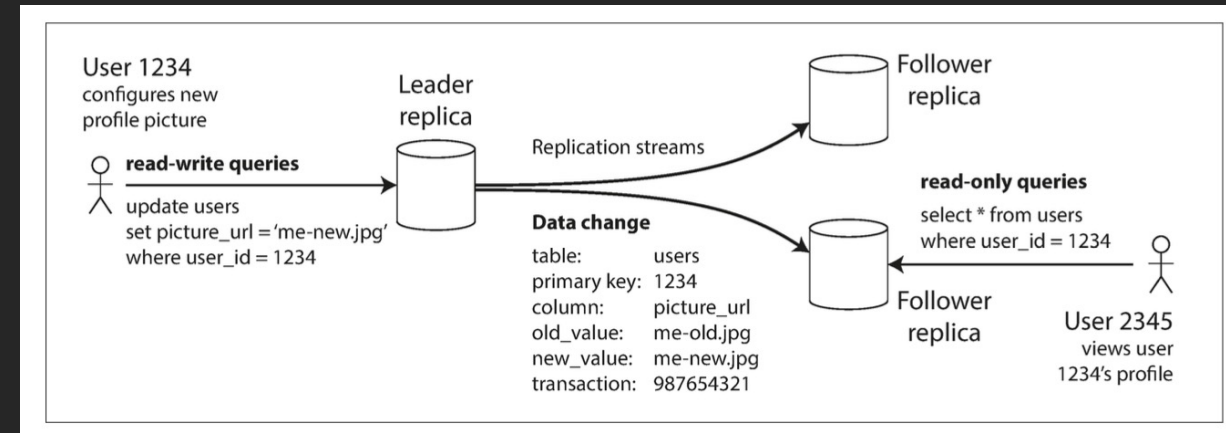
Replication

Replication

- Each node stores a copy of the data (i.e., replica)
 - Can reduce latency keeping data geographically close to the users
 - Increase availability allowing the system to continue working even if some of its parts have failed
 - Increase performance scaling out the number of machines that can serve read queries
 - Your data has to be small enough to be stored in a single machine
 - Write operations involve updating all the copies of the data
-

Leaders and followers

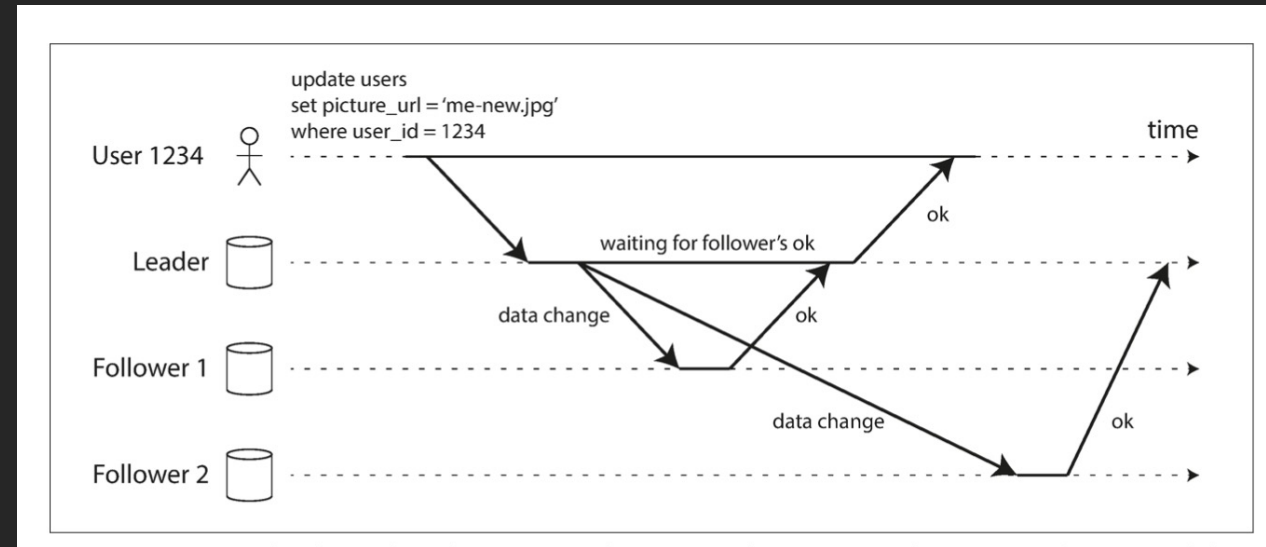
- One of the replicas is designated the leader
Write operations are sent to a leader node which writes it to its local storage
- The leader will send the update to the other replicas (i.e., followers) as part of a replication log.
The followers will apply the same changes, in the same order, to their local copy of the data
- Read queries can be sent to any replica while writing operations can only be sent to the leader
- Built-in to many DBMSs like PostgreSQL, MongoDB, and MySQL



Synchronous replication

- The leader will wait for the followers to confirm they received the write operation before reporting success
- Should they wait for all the followers to confirm success?
- Synchronous replication often means at least one follower is synchronous

If the synchronous follower becomes unavailable or slow, one of the asynchronous followers is made synchronous (AKA semi-synchronous)



Adding new followers

- Simply copying the data ?

The data is very dynamic

- Locking the database?

Locking the database for writes is not ideal either

Adding new followers

- Take a snapshot of the leader's database at some point in time
 - Copy the snapshot to the new follower node.
 - The follower requests all the data changes that have happened since the snapshot was taken.
 - When the follower has processed the backlog of data changes, we say it has *caught up*.
-

Recovering from failures

- Follower failure

From its logs, it knows the last processed transaction

Connect to the leader and request the data changes

- Leader failure (failover)

One of the followers will be promoted to the new leader

- Often the replica with the most up-to-date data changes from the old leader

Many problems can arise

- Split brain
-

Replication lag

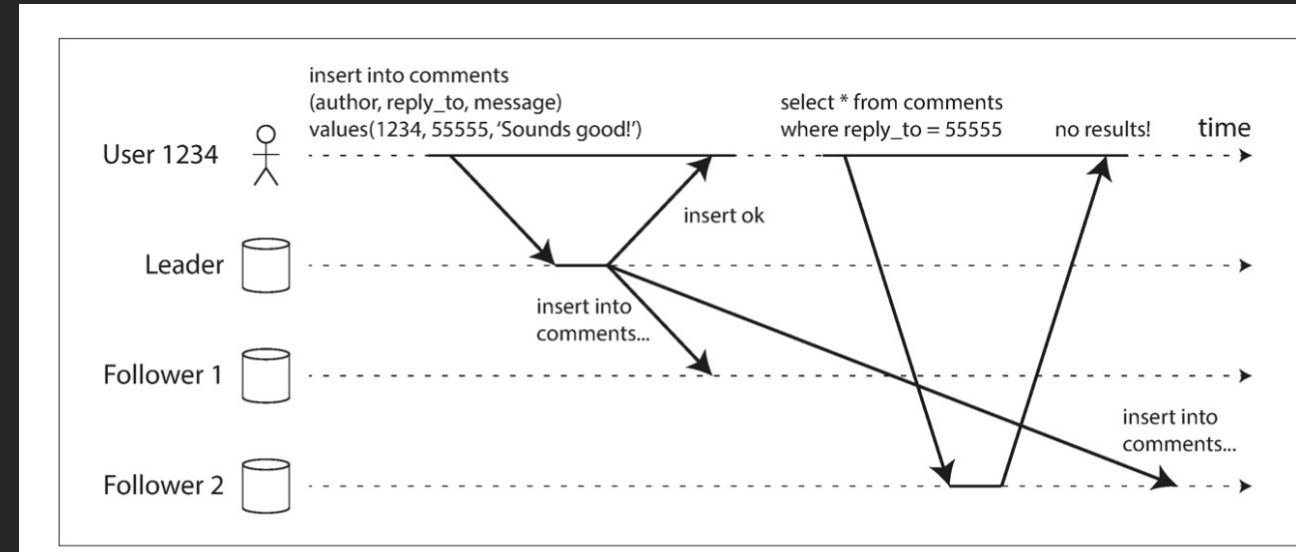
- Read- only queries can go to any replica
 - If an application reads from an *asynchronous* follower, it may see out-dated information
 - Many problems can arise from that
-

The lost data

- if the user views the data shortly after making a write, the new data may not yet have reached the replica.

read-your-writes consistency

- if the user reloads the page, they will always see any updates they submitted themselves.



Backwards data

- if a user makes several reads from different replicas it may see things *moving backward in time*
- *Monotonic reads*
Make users always makes their reads from the same replica

